

Seje



dr. Matej Šprogar

HTTP zahteve niso povezane

Spletni strežnik obravnava vsako zahtevo tako, kot da je prišla od *novega* klienta - HTTP ne omogoča beleženja zgodovine zahtev posameznega uporabnika.

Tudi če je že odgovoril nekemu klientu, protokol tega ne beleži, kaj šele, kakšen je bil odgovor...

Če želi strežnik povezati zahteve *istega* uporabnika skozi čas, mora sam uporabiti mehanizem, ki nekako združi podatke vseh zahtev, kar mu omogoča odgovore čez več zahtev istega klienta, ne zgolj na zadnjo prejeto.

Dve možni rešitvi

Na zahtevo lahko odgovorimo šele, ko imamo na razpolago VSE potrebne podatke, tudi tiste iz morebiti *preteklih* zahtev:

- A. *V vsaki zahtevi pričakujemo **vse potrebne** podatke!*
Podatke, ki jih je klient poslal v prvi zahtevi, "skrijemo" v prvi odgovor in jih tako spet sprejmemo v "drugi" zahtevi... Isti podatek se večkrat prenaša po mreži, je pa vsaka zahteva neodvisna celota.
- B. *V vsaki zahtevi potrebujemo unikatno **oznako** klienta!*
Namesto prenašanja že prenešenih podatkov prenašamo zgolj kratko oznako, ki na server strani služi kot ključ za dostop do **sejnih podatkov** tega klienta.

Sejni podatki so torej lahko na...

1. Klientu / v sporočilih

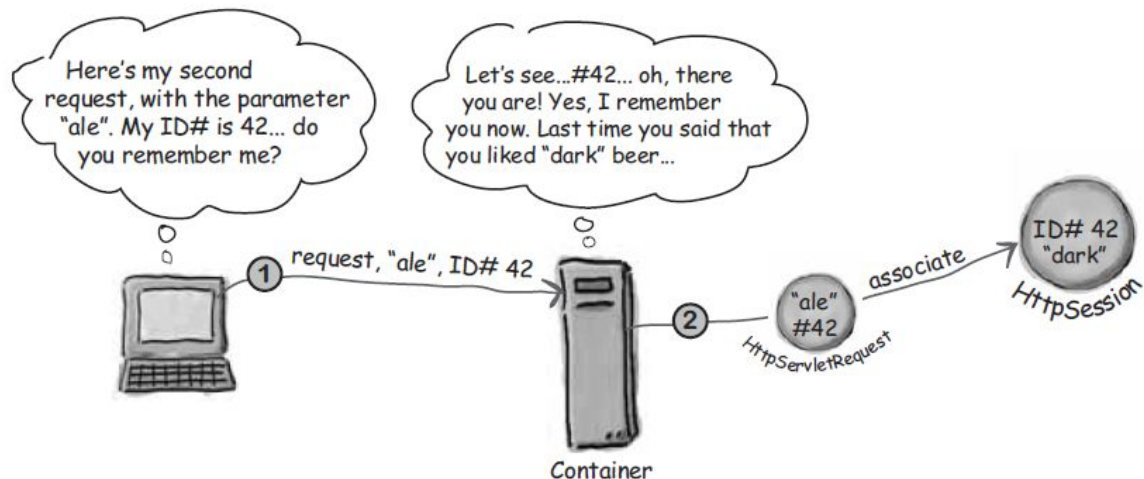
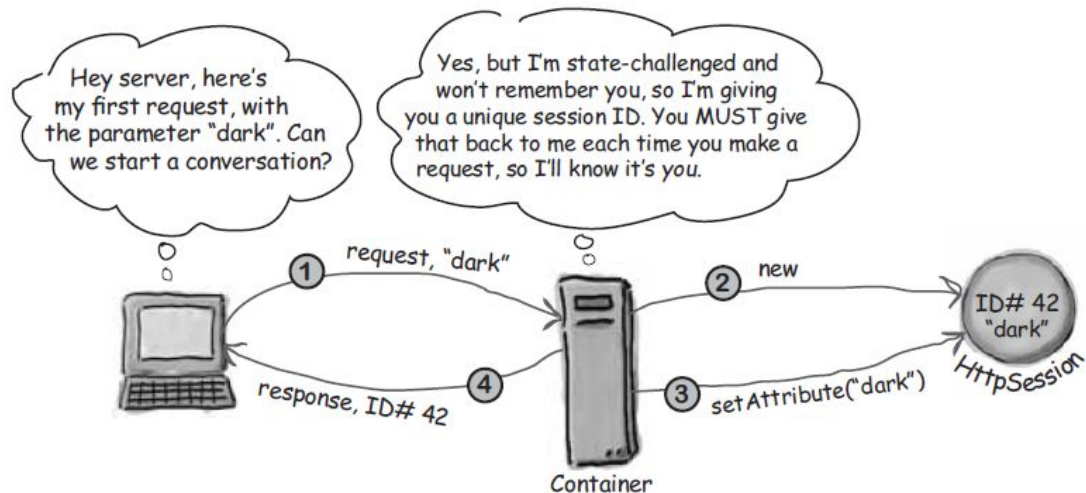
Pri vsaki zahtevi in odgovoru je potrebno prenašati **vse potrebne** podatke - težave z varnostjo in integriteto podatkov, počasno za večje količine podatkov; izvedljivo s pisanjem podatkov v URL, v piškote, v skrita polja v formi ali v objekte v bogatem odjemalcu (rich-client)

2. Strežniku

Komunikacija obsega zgolj **identifikator seje** in **tekoče** podatke, ostali sejni podatki so naloženi/shranjeni v:

- a. *pomnilniku strežnika*; hitro delovanje in implementacija.
- b. *podatkovni bazi* na tem istem ali drugem strežniku; podatki so najbolj varni, problem sinhronizacije s podatki strežnika, težave z *izolacijo* podatkov.

Uporaba sejne oznake



Identifikacija klienta

- Vsak klient mora imeti unikatno sejno oznako; to pomeni, da lahko sejno oznako generira zgolj strežnik.
- *Prva* zahteva klienta *ne more* vsebovati sejne oznake; to je za strežnik znak, da ima novega klienta in da mu mora v odgovoru sporočiti njemu dodeljeno oznako.
- Če je v zahtevi vključena oznaka seje je to znak, da je klient v preteklosti že dostopal do strežnika in želi nadaljevati s komunikacijo.

Prenašanje sejnih oznak

1. z integracijo v URL (odsvetovana praksa)

Generiran odgovor je napisan tako, da vsak URL vsebuje tudi sejno oznako, ki bo tako vsebovana v naslednji klientovi zahtevi strežniku.

Integracijo URL-jev mora izrecno opraviti programer!

2. s piškoti - *Cookies* (priporočena rešitev)

Strežnik v HTTP glavi odgovora nastavi polje *Set-Cookie*, zato brskalnik ustvari piškot z oznako seje. Brskalnik v glavo ustrezne HTTP zahteve *avtomatsko* pripne tudi piškot sejne oznake, ki je namenjen temu strežniku.

Sejne piškote avtomatsko pošilja browser!

1. Sejna oznaka v URL (nevarno)

- je zasilni mehanizem
- možna *varnostna luknja* (*link sharing, session fixation...*)
- Če **vsi** URLji v odgovoru niso oblikovani z mislijo na dodajanje sejne oznake, je vodenje sej *nemogoče*:

potrebno je “ročno” ustvarjanje *vseh* linkov:

```
out.write("<a href=\"\" + rewriteURL(response, url) + \"\">link</a>");
```

```
<a href="<%= rewriteURL(response, url) %>">link</a>
```

- Implementacija ni standardizirana - sejna oznaka bo v URL dodana na “nek” način, na primer:

```
http://localhost:8080/app/doit;sid=1700C994F991BE0E38...
```

- Robusten, a nevaren mehanizem, ki zahteva skrbnost!

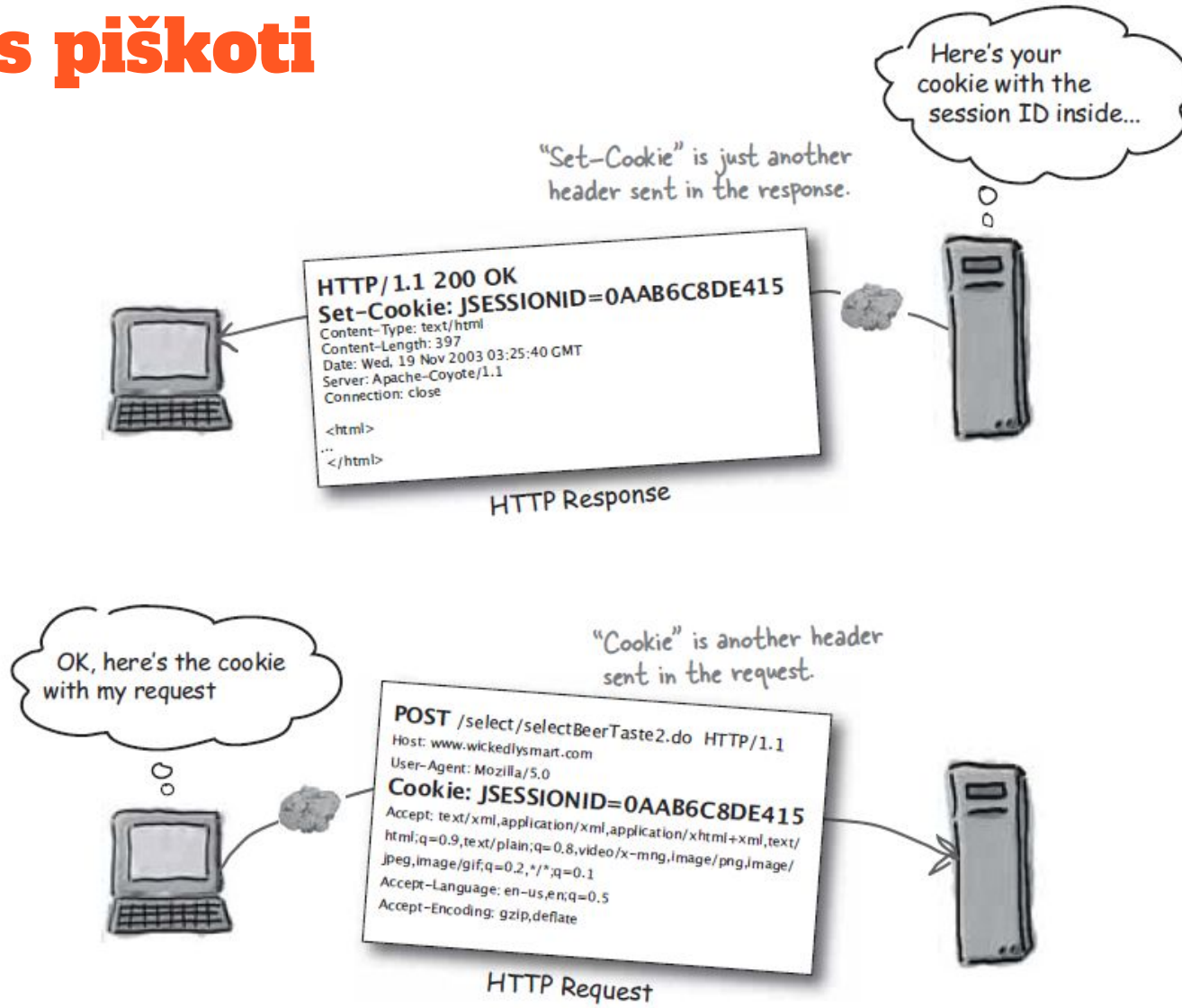
Seja s sejno oznako v URL



2. Piškoti (priporočljivo)

- Piškoti so splošna tekstovna informacija:
`<ime,vrednost,domena,pot,Max-Age,Secure,HttpOnly,SameSite>`
- **Sejni piškot** nosi *sejno* oznako in ga vzpostavi strežnik avtomatsko*, ostale piškote ustvarimo ročno
- V zaglavju vsake HTTP zahteve pošlje browser avtomatsko **vse** *ustrezne* (domena & pot) piškote na strežnik
- Najenostavnejša identifikacija brskalnika
- Možne težave s privatnostjo, ker lahko tretji strežnik sklepa o uporabnikovih zahtevah
- Enostaven, a ne vedno razpoložljiv mehanizem!

Seja s piškoti



Domena in pot piškotov

- Vsak piškot je označen z domeno in potjo
- Brskalnik pošlje ob zahtevi le tiste piškote, ki se ujemajo z obema: s strežniško domeno in potjo
- Privzeta piškotova domena je strežnik, ki je poslal piškot. Če delamo z več strežniki, moramo nastaviti domene tako, da klient vrača piškote na vse strežnike, ki procesirajo zahtevo.

Staranje piškotov

- Vsakemu piškotu je mogoče nastaviti datum zapadlosti (*Expires*) in/ali čas trajanja (*Max-Age*) - zastarane piškote bo brskalnik izbrisal avtomatsko
- Z nastavitvijo časa trajanja piškota povemo, koliko sekund naj brskalnik ohranja piškot:
 - če je trajanje dovolj veliko pozitivno število, bo piškot preživel tudi morebitno zapiranje brskalnika
 - če je trajanje 0 ali negativno število, bo brskalnik takoj izbrisal piškot
- Brisanje (starih) piškotov se izvede z nastavitvijo njihovega trajanja na 0 sekund
- Če sta nastavljena tako *Max-Age* kot *Expires*, prevlada *Max-Age*

Atributi piškota: SameSite, Secure, HttpOnly

SameSite atribut natančneje določa pošiljanje piškotov ob nalaganju polne vsebine strani - ali v okviru prve strani ali tudi tretje strani (Strict | [Lax] | None).

Secure atribut določa, da se lahko piškot pošilja izključno preko **https** enkriptiranih povezav.

HttpOnly atribut povzroči, da JavaScript v brskalniku nima dostopa do tega piškota; uporablja ga lahko izključno le http(s), kar je mnogo varnejše.

Tipi in trajnost piškotov

- First party cookie
 - Je piškotek, ki ga nastavi spletna stran, ki je trenutno odprta v brskalniku
 - Nastavi ga ista domena, kot jo uporabnik obiskuje neposredno.
- Third party cookie (onemogočeni v 2025?)
 - Je piškotek, ki ga nastavi druga spletna stran (domena), ki je vdelana v trenutno odprto stran – na primer preko oglasov, vtičnikov, iframov ali zunanjih skript (npr. youtube, google maps...).
 - Nastavi ga domena, ki ni enaka trenutni obiskanosti.

Trajnost:

- Session cookies - obstajajo za čas ene seje; vklopimo *HttpOnly*, *SameSite=strict*, *Secure*!
- Persistent cookies - obstajajo do zastaranja (web analytics...)

Izklopljeni piškotki?

Strežnik ne more enostavno ugotoviti, ali brskalnik omogoča piškotke. Lahko poskusi s *Set-Cookie*, vendar bo do nove zahteve že "pozabil", da pričakuje piškotek... Tudi če izvede *redirect* brskalnika na poseben naslov, rezerviran za brskalnike brez piškotov, bo to znanje spet takoj izpuhtelo...

Brez piškotkov ni trajnega spomina, zato je za trajnostno rešitev potrebno poseči po drugih tehnikah, kot so:

- localStorage / sessionStorage (nevarno, ker ima dostop JS)
- URL parametri (nevarno, ker ga je mogoče prestreči ali nenamerno razkriti)
- zahteva po omogočenih piškotkih (slabo, ker izgubimo stranko)

Lastnosti sej

- Če seje vzdržujemo s piškoti, je sejna oznaka vezana na browser, če pa sejne oznake prenašamo v URLju, pa lahko imamo hkrati odprtih več sej v istem brskalniku
- Če uporabljamo sejne oznake, potem strežnik vzdržuje po en “sejni objekt” za vsako sejno oznako
- Neaktivne seje zahtevajo strežniške vire, zato jih je smiselno brisati:

Seja na strežniku zapade, če:

- a. ji poteče čas trajanja
- b. je izrecno zahtevan preklic seje (npr. pri odjavi)
- c. se zapre celotna aplikacija/strežnik

Pozor

URL rewriting lahko preoblikuje URL *interno* na strežniku:

`http://www.xxx.com/working -> http://www.xxx.com/nicer`

URL redirekcija preusmeri *browser* na drugi naslov:

`http://www.old.com/alfa -> http://www.new.com/beta`

URL forwarding pomeni, da strežnik *interno* posreduje zahtevo naprej, počaka na rezultate in jih vrne, kot da jih je sam proizvedel. Brskalnik tega *ne opazi*.

URL encoding je zapis sicer rezerviranih znakov:

`Halo "svet"! -> Halo+%22svet%22%21`